

Architecture Documentation

DEVision - Job Manager Subsystem

Samson Squad

November 2025

Team Members

Tran Kiem Phuc - s3927187

To Nhat Duy - s3979932

Pham Quang Huy - s3926751

Tran Minh Triet - s3979253

Vo Phuc Duy Nhat - s3868763

Table of Contents

1. Introduction.....	4
2. Squad Data Model.....	4
2.1. Data Model	4
3. Squad Container Diagram.....	5
3.1. Job Applicant Subsystem Overview.....	5
3.2. Job Applicant Frontend Container Diagram.....	6
3.3. Job Applicant Backend Container Diagram	7
3.4. Job Manager Frontend Container Diagram.....	8
3.5. Job Manager Backend Container Diagram	12
4. Backend C4 Component Diagram.....	13
4.1. Backend Job Post C4 Component Diagram.....	13
4.2. Backend Company (user) C4 Component Diagram	14
4.3. Backend Subscription C4 Component Diagram	15
4.4. Backend User Authentication C4 Component Diagram	15
4.5. Backend Payment C4 Component Diagram	16
5. Frontend Component Diagram	17
5.1. Frontend Job Post Feature C4 Component Diagram	17
5.2. Frontend Login/Registration Feature C4 Component Diagram	18
5.3. Frontend Profile Feature C4 Component Diagram	20
5.4. Frontend Subscription C4 Component Diagram.....	21
5.5. Frontend Applicant Search C4 Component Diagram	23
6. Architecture Rationale.....	24
6.1. Data Model Justification.....	24
6.2. Frontend Architecture Justification	26

6.3. Backend Architecture Justification..... 28

7. Conclusion 31

1. Introduction

This document provides a complete architectural overview of the Job Manager (JM) and Job Applicant (JA) subsystem within the DEVision platform. It includes the system data model, frontend and backend container diagrams, component diagrams, and the architectural rationale supporting key quality attributes. The goal of this documentation is to ensure all team members understand the system architecture and provide clarity by presenting how the system is structured, how its components interact, and how each design aligns with the overall DEVision architecture.

The subsystem supports the following key features in the SRS:

- User authentication and role-based access for applicants, companies, and admins
- Applicant profile creation and management
- Company registration, profile, and subscription management
- Job posting creation and management
- Applicant tracking workflow (application, review, shortlisting, status update)
- Notification delivery for both applicants and companies
-

Our architecture follows:

- **Frontend Architecture Style:** React-based Modularized Component Architecture
- **Backend Architecture Style:** Spring Boot Modular Monolith with domain-grouped modules

2. Squad Data Model

2.1. Data Model

[Samson Squad Data Model Diagram](#)

The JM/JA data model is designed to support the core functionalities required by the recruitment workflow. The model captures the information needed for applicant profiles, company profiles, job posts, subscriptions, payments, authentication, and notifications.

The model supports the SRS requirements in the following ways:

- **Applicant Features:** The ApplicantProfile entity stores personal details, experience levels, and creation settings. The attributes, such as skills, education, and work experience, allow applicants to build a complete professional portfolio. This data is used by companies when reviewing job applications.
- **Company Features:** Company and CompanyMedia capture the organization's identity, branding, and contact details. These details support the company's onboarding, profile display, and public visibility.
- **Job Posting & Application Workflow:** JobPost and JobApplication form the core of the recruitment flow. Companies create job posts; applicants apply, and the system tracks progress through various application statuses.
- **Subscription & Payment:** Subscription and PaymentTransaction store subscription tier, billing cycles, and transaction details. This allows the system to enforce posting limits and unlock paid features for companies.
- **Authentication:** UserAuth and CompanyAuth centralize login credentials for applicants, companies, and admins. The authentication supports role-based features and provides tokens for frontend squads.
- **Notification:** Notification and EventID support system-wide alerts for actions such as job application status updates, subscription renewals, and system messages.

3. Squad Container Diagram

3.1. Job Applicant Subsystem Overview

This subsystem is designed for individuals seeking jobs in the Computer Science domain. It is responsible for all applicant-facing functionalities, including:

- Account registration
- Login & authentication
- Profile creation and management
- Job search & filtering
- Job application submission
- Application status tracking
- Premium subscription and real-time notifications

To support these features, the subsystem is built using a **Modular Frontend** and a **Microservices Backend**, ensuring maintainability, scalability, and a clean separation of

concerns. It also utilizes **Kafka** for event-driven premium notifications (such as new job alerts) and stores applicant-related data using **MongoDB**.

Connection Through API Gateway

The Job Applicant Subsystem does not communicate with other subsystems directly. Instead, all communication is routed through the API Gateway, including:

- Fetching public job listings
- Submitting job applications
- Searching jobs
- Synchronizing application status

The API Gateway is responsible for request routing, authentication and token validation, cross-system communication, and hiding internal service endpoints from external access. This ensures that subsystem interactions remain secure, consistent, and well-structured.

3.2. Job Applicant Frontend Container Diagram

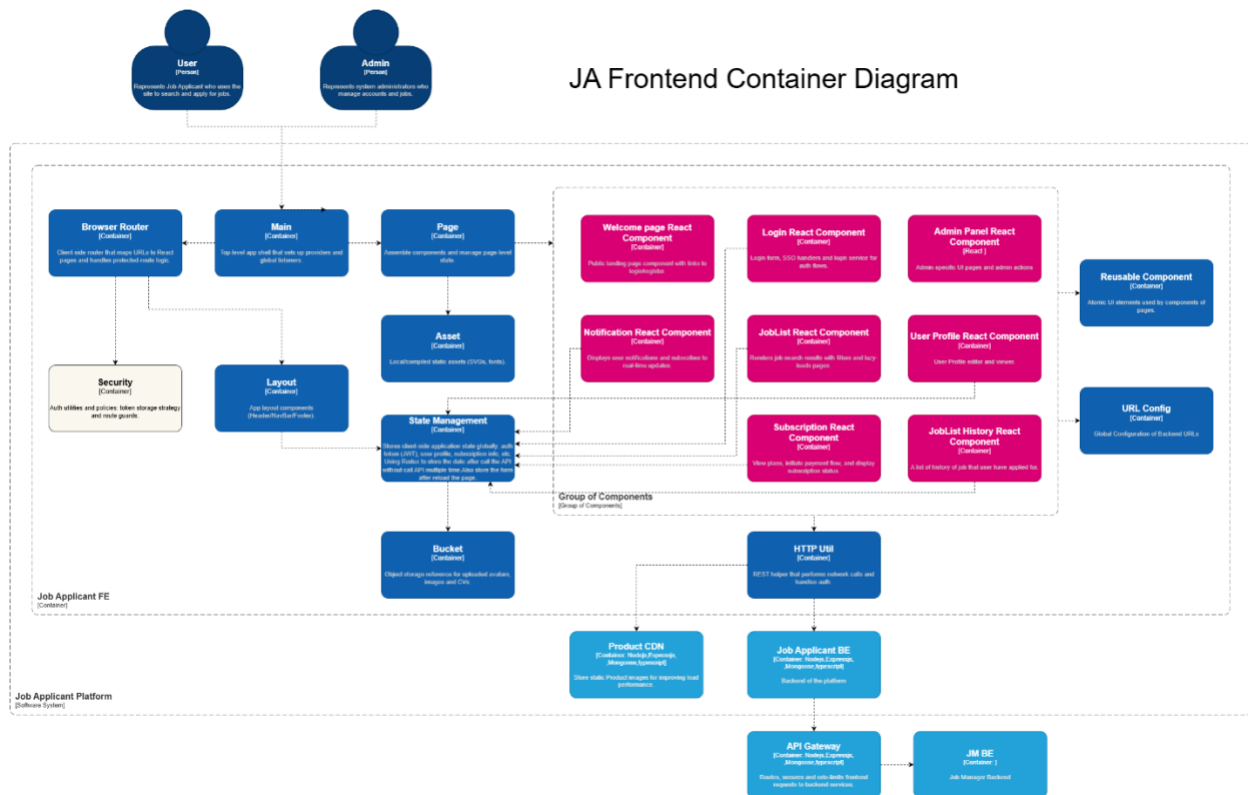


Figure 1: Job Applicant Modular Frontend Container Diagram

The backend is structured as multiple independent microservices:

- Application Service
- Auth Service
- Profile Service
- Search Proxy Service
- Job Posts Service
- Application Service
- Matching Profile Service
- Subscription Service
- Notification Consumer (Kafka)

Additional architectural characteristics include:

- Implements **SOLID principles**, enabling loosely coupled, testable, and maintainable code.
- Each service runs inside a **dedicated Docker container** for easier deployment and scaling.
- Uses **Service Discovery**, allowing microservices to register and communicate dynamically without requiring hardcoded URLs.

3.4. Job Manager Frontend Container Diagram

[Frontend C4 Container Diagram Link](#)

Architecture Overview: The Job Manager frontend is divided into container-level modules that manage high-level concerns (security, routing, global state), and feature-level modules that deliver the functionality used by company users. Containers such as Browser Router, Security, State Management, and Layout form the backbone of the system, while feature containers such as Job Post Feature, Applicant Search, and Subscription & Payment implement the core business functions. All communications with the backend occur through a dedicated HTTP Unit, which encapsulates API URLs, headers, token injection, and error handling. This avoids duplication and ensures consistent communication across all modules.

- **Technology Stack:**
- **Frontend:** React
- **Architecture Style:** Modularized Component Architecture
- **Language:** JavaScript / TypeScript
- **Routing:** React Router
- **State Management:** React Context API

- **HTTP Client:** Custom HTTP Unit
- **Persistent Storage:** LocalStorage for subscription/payment state
- **UI Utilities:** Bootstrap library (buttons, inputs, modals, etc.)

The **core containers** are listed below:

- **Browser Router:** Manages all navigation throughout the Job Manager system, defining route-to-page mapping for the following:
 - Job Post Management
 - Applicant Search
 - Subscription and Billing
 - Company Profile
 - Login/Registration

Browser Router works with the Security container to block access to protected routes and is displayed based on the URL.

- **Security Container:** Protects all Job Manager pages from unauthorized access:
 - Validates JSON Web Token (JWT) tokens stored during login
 - Checks that the user's role is 'company' before allowing access
 - Redirects unauthenticated users to Login/Registration
 - Prevents access to job posting, applicant search, and subscription pages without authentication.
- **State Management Container:** Provides centralized store for global data that is used across multiple modules, which maintains the following:
 - Auth token and user role
 - Company profile information
 - Job posts and applicant information
 - Subscription tier and billing state
 - Filter and search presets

All the feature modules will read and update data through the state management layer to ensure data consistency across pages.

- **Layout Container:** Shared visual structure of the system, which includes:
 - Navigation bar
 - Header and toolbar
 - Grid layout

- panels
- Shared UI elements

Every page in the Job Manager system is wrapped inside this layout to maintain a consistent user experience.

- **Main Container:** The root for the layout and routing controller. The central point where global cross-page logic is handled, and what it does is:
 - Load appropriate page module
 - Integrates layout components
 - Applies security check
- **Page Container:** A wrapper that configures each page's UI structure and allows child components to mount feature logic. It groups the following pages:
 - Job Posts (CRUD)
 - Subscription
 - Applicant Search
 - Profile
 - Subscription
 - Registration
- **HTTP Unit Container:** Utilizing Spring Boot for frontend service module communication with backend, its functions include:
 - Injecting JWT tokens into headers
 - Handles redirection on unauthorized requests
 - Calling REST endpoints for job posts, company data, applicants, and subscriptions.
 - Managing centralized error responses

This design would avoid fetching code across modules and ensure consistent API behavior.

- **URL Config Container:** Stores the mapping of frontend routes and backend API endpoints:
 - /company/profile
 - /company/jobs
 - /company/subscription
 - /applicant/search

This container ensures that all modules use a single source for navigation and API routing.

- **Cart State Storage Container:** Used primarily by the Subscription & Payment feature which persists:
 - Subscription selections
 - Plan tiers
 - Payment attempts

The module is stored inside LocalStorage, which ensures that the workflow continues even when the page is refreshed.

- **API Gateway:**

The **API Gateway** acts as the entry point for all API requests. It serves to **route** requests from the frontend to the appropriate backend services, such as **Job Post Backend**, **Applicant Search Backend**, and **Job Applicant Backend (JA BE)**. The **API Gateway** ensures that requests are authenticated and properly routed, allowing for better **scalability** and **security** by centralizing API request handling.

 - **For example**, when the **Job Post Feature** submits a job post request, the **API Gateway** routes it to the appropriate **Job Post Backend API**.
- **Job Applicant Backend (JA BE):**

The **JA BE** manages applicant-related data and services. This backend handles job applicant data storage, queries for applicant profiles, and integrates with the **Applicant Search** feature. The **frontend** interacts with this service via the **API Gateway**, ensuring secure and efficient data exchange.

 - **For example**, when a company user searches for applicants in the **Applicant Search** feature, the request is routed through the **API Gateway** to the **JA BE**, which returns the relevant data to the frontend.

Communication with Backend: The JM frontend communicates with the backend through RESTful HTTPS APIs, which are wrapped inside the HTTP Unit. The requests include:

- authorization headers contain the JWT token
- DTOs for job posts, applicants, company profiles, and subscription
- Role validation performed by the backend
- The **API Gateway** ensures requests are routed properly, providing an extra layer of security and scalability by managing routes and API endpoints centrally.

- **Backend services**, like the **Job Post Backend**, **Applicant Search Backend**, and **JA BE**, handle data-related tasks and communicate with the frontend through the **API Gateway**.

All of the responses are then passed to State Management or would be passed directly to feature components.

3.5. Job Manager Backend Container Diagram

Back-end C4 Container Diagram

Architecture Overview: The Job Manager backend adopts a Modular Monolith architecture using Spring Boot, where each major business capability is implemented as an independent backend module within a shared application. Incoming requests from the front end first pass through the Filter and Security containers, where request preprocessing, token validation, and authorization checks occur. Once validated, requests are routed to the appropriate module: the Auth module handles authentication and registration, the Company module manages company profiles and employer operations, the JobPost module processes job posting workflows, and the Subscription and Payment modules handle subscription features and premium plan payments. These modules interact with a centralized MongoDB database, which stores authentication data, job posts, company information, subscriptions, and payment records.

The backend also integrates with external services to support advanced functionality. The Google Auth Service is used for third-party authentication flows, while Stripe processes premium subscription payments. Additionally, the Job Manager backend communicates with the DEVision Job Application subsystem to retrieve applicant profiles and update their application statuses. This modular backend structure ensures clear separation of responsibilities, secure request handling, and efficient interaction between internal modules and external systems while maintaining consistency within a single codebase and database.

For the backend container diagram of the Job Manager subsystem, the following technology stack will be used:

- **Language and framework:** Java, Spring Boot
- **Deployment:** Docker

- **Payment:** Stripe
- **Auth:** Google OAuth
- **Database:** MongoDB

A modular monolithic architecture is used, meaning that the backend is split into separate modules that can be deployed together as a single unit. The core modules in the container diagram have the following purposes:

- **Filter:** Preprocesses requests such as CORS or routing and forwards them to the internal modules.
- **Security:** Token validation, generation, cryptography, and global security utilities.
- **Auth:** Authentication and registration, and SSO (Single Sign-On). Also works with the Security module to generate tokens.
- **Company:** Manage their own account and profile, search for job applicants and call the JobPost module to create job postings.
- **JobPost:** Create and manage job postings.
- **Subscription:** Handle premium company subscriptions, their lifecycle (when it starts/expires), and subscription status.
- **Payment:** Handles payments for subscriptions via Stripe

As for communication, the backend communicates with other subsystems as follows:

- **Frontend:** The frontend interacts with the backend via HTTPS REST through the Filter gateway
- **Internally:** Direct function calls between modules
- **Database:** Read and write to the shared database
- **External systems** (Stripe, Google OAuth, Job Applicant Backend): Calls via HTTPS REST

4. Backend C4 Component Diagram

4.1. Backend Job Post C4 Component Diagram

[Back-end C4 Component Diagram \(Job Post module\)](#)

Overview: This diagram shows how different layers of Job Post module work together to handle job post operation requests from front end and are managed by Company module inside the Job Manager app. They also handle requests from Job Applicant app and send back Job Post listings DTOs.

Interaction:

- Filter acts as the end point for the Controller layer, making sure correct requests are sent to be processed.
- **For external components** (such as Job Application and Company module) to access data inside, they must go through External Data packages and first send requests to the External interface, then receive External DTOs from said interface.
- **For internal layers:** Each layer above can only access the layer below, not vice versa. So, to enable that, we have an Internal interface combined with its own DTOs, allowing cross-layer communication between the Controller and Service.
- **For data:** Object mapper is used extensively to transform Java objects into JSON to be put into our database.

4.2. Backend Company (user) C4 Component Diagram

[Back-end C4 Component Diagram \(Company module\)](#)

Overview: This diagram shows how different layers of Company module work together to process Company operations requests from front end and manage Job Post module inside the Job Manager app. They also handle requests from Job Applicant app and send back Company Profile DTOs.

Interaction:

- Filter acts as the endpoint for Controller layer, making sure the correct requests are sent to be processed.
- For external components (such as Job Application and Job Post module) to access data inside, they must first send requests to the External interface, then receive External DTOs from said interface.
- For usage of outside data from Job Post Module, DTOs must be sent to the Controllers to be passed down to lower-level layers.

- For internal layers: Each layer above can only access the layer below, not vice versa. So, to enable that, we have an Internal interface combined with its own DTOs, allowing cross-layer communication between the Controller and the Service.
- For data: Object mapper is used extensively to transform Java objects into JSONs to be put into our database.

4.3. Backend Subscription C4 Component Diagram

[Back-end C4 Component Diagram \(Subscription module\)](#)

Overview: This diagram shows how different layers of Subscription module work together to handle company's subscription operations' requests from front end inside the Job Manager app.

Interaction:

- The filter acts as the endpoint for the Controller layer, making sure the correct requests are sent to be processed.
- For external components to access data inside, they must first send requests to the External interface, then receive External DTOs from said interface.
- For internal layers: Each layer above can only access the layer below, not vice versa. To enable that, we have an Internal interface combined with its own DTOs, allowing cross-layer communication between the Controller and Service.
- For data: Object mapper is used extensively to transform Java objects into JSON to be put into our database.

4.4. Backend User Authentication C4 Component Diagram

[Back-end C4 Component Diagram \(Auth module\)](#)

Overview: This diagram shows how different layers of Auth module work together to process authentication requests from front end and provide tokens to security modules inside the Job Manager app. They also handle requests from Job Applicant app and send back permission to work within the app too.

Interaction:

- Filter acts as the endpoint for the Controller layer, making sure the correct requests are sent to be processed.
- For external components (such as Job Application & Company module) to access data inside, they must first send requests to the External interface, then receive External DTOs from said interface to gain permission to work within Job Manager App.
- This external interface also uses outside services such as the Security module for their token and Google Auth Services, Microsoft Auth Services for their 3rd party SSO.
- For internal layers: Each layer above can only access the layer below, not vice versa. So, to enable that, we have an Internal interface combined with its own DTOs, allowing cross-layer communication between the Controller and Service.
- For data: Object mapper is used extensively to transform Java objects into JSON to be put into our database.

4.5. Backend Payment C4 Component Diagram

[Back-end C4 Component Diagram \(Payment module\)](#)

Overview: This diagram shows how different layers of Payment module work together to process payment requests from the front end through usage of Stripe API. They also handle requests from the Job Applicant app and allow them to use Payment APU as well if needed.

Interaction:

- Filter acts as the endpoint for the Controller layer, making sure the correct requests are sent to be processed.
- For external components (such as Job Application) to access data inside, they must first send requests to the External interface, then receive External DTOs from said interface.
- Payment's External Interface also uses Stripe API, which requires us to call the API.
- For internal layers: Each layer above can only access the layer below, not vice versa. To enable that, we have an Internal interface combined with its own DTOs, allowing cross-layer communication between the Controller and the Service by sending necessary DTOs to the requested layers.

- For data: Object mapper is used extensively to transform Java objects into JSON to be put into our database.

5. Frontend Component Diagram

5.1. Frontend Job Post Feature C4 Component Diagram

Frontend C4 Modularized Job Post Feature

Overview:

The **Job Post Feature** is built using a modular structure that separates UI elements and business logic. The **Job Post Form** component allows users to fill in job details such as title, description, and location, while the **Job Post Quick Actions** provides buttons for publishing, closing, duplicating, or deleting job posts. These UI components interact with **useJobPostHook**, which manages the core logic of job posting, including state validation, triggering CRUD operations, and managing state transitions.

Purpose:

The Job Post Feature allows company users to manage job postings on the job Manager platform. It enables actions such as creating, updating, publishing, duplicating, or deleting job posts. The feature is modularized into smaller components to ensure clean separation of UI and logic. The Job Post Form handles the input fields for job details, and the Job Post Quick Actions manages tasks like publishing and closing jobs. This structure ensures flexibility and maintainability when adding or modifying job-related actions.

Incoming communication:

1. Authentication and User Info:

- The **Job Post Feature** first ensures that the user has the correct permission to post a job. The **Security** container verified that the user is authorized to create or manage job posts.
- **State Management** retrieves and stores the current user's session and authentication status, making sure only valid users can interact with the job posting features.

2. Backend Communication:

- **useJobPostHook** handles the logic for validating user inputs, preparing the job post data, and triggering actions

Outgoing Communication:

1. User Submits Job Post Details:

- When a user submits the job post, the **Job Post Form** sends the data to **useJobPostHook**, which then passes it to the **Job Post Service** via backend **REST APIs**.
- The **Job Post Service** communicates with the backend to create, update, publish, or close job posts. This service handles all backend logic, ensuring the data is saved properly.

2. Backend Validation and Response:

- The **Job Post Service** queries the backend and retrieves a response, which is then parsed and sent back to the frontend.
- If the job post action is successful (e.g., job post published), **State Management** is updated to reflect the new job status

3. UI update and Routing:

- **State Management** stores the updated job post status (e.g., publish or closed) and makes it available to other parts of the system.
- **Reusable Components** such as buttons and notifications are updated, and the user is redirected to the appropriate page.

Interaction Flow:

- UI collects job details -> **useJobPostHook** validates and sends data -> **Job Post Service** communicates with backend -> **URL Config** manages routes -> result returned and stored in **State Management** -> user routed to the next page.

5.2. Frontend Login/Registration Feature C4 Component Diagram

[Frontend C4 Modularized Login Feature](#)

Overview: The Login Feature is structured to keep the UI clean and the logic fully separated. The **Login Form component** displays all required input fields for a company user to log into DEvision, such as email, password, country, and address. For users who prefer external authentication, the **Social Login component** provides **Google login** as an alternative. Both of these UI components hand their input values to the **Login Hook**, which

manages all core login logic. The hook checks credential format, validates user input, manages loading/error states, and prepares the request payload.

Purpose: The Login Feature allows users to securely authenticate and access the Job Manager platform. It provides **two login options**: a traditional form for entering credentials (email, password, etc.) and a social login option using Google for convenience. This feature manages user authentication, ensuring that only valid users can access their personalized information and perform actions within the system.

Incoming Communication:

1. Authentication and user Info:

- The **Login Feature** first checks user authentication through the Security container to ensure only authorized access to login functionalities.
- **Browser Router** directs the user to the **Login Page**, while **State Management** ensures the user's current session and authentication state are available to pre-fill or validate fields

2. Backend Communication:

- The **Login Hook** manages the core login process, where it validates the user input and prepares the payload. It then communicates with the backend via the **Login Service** to validate credentials.
- All data is passed securely through **URL Config**, which manages URL paths for login routes and ensures the correct routing and authentication headers are in place.

Outgoing Communication:

1. User Submits Credentials:

- When the user submits their login credentials, the **Login Form** sends the data to **Login Hook**, which forwards the request to the **Login Service** via backend REST APIs.
- The **Login Service** handles the backend logic, authenticating the user, parsing responses, and checking for success or errors.

2. Backend Validation and Response:

- The **Login Service** queries the authentication API and retrieves the response, which is then processed and returned to the frontend.

- If successful, the user's session is updated in **State Management**, and the user is redirected to the appropriate page.

3. UI update and Routing:

- The **State Management** container stores the authenticated user's details, and **Assets** provide the necessary UI elements like icons and images.
- After a successful login, **Reusable Components** such as buttons and inputs are updated across the system, and the user is routed to the next page.

Interaction Flow:

- UI collects credentials -> **Login Hook** validates and sends data -> **Login Service** communicates with backend -> **URL Config** manages routes -> result returned and stored in **State Management** -> user routed to the next page.

5.3. Frontend Profile Feature C4 Component Diagram

Frontend C4 Modularized Profile Feature

Overview: The diagram illustrates the internal structure, data flow, and interactions involved in managing the company profile within the Job Manager frontend. This module supports the company user's ability to view and update their organization's information, branding assets, industry category, and contact data. The diagram shows how frontend features work together with backend services such as validation and media storage.

Purpose: The component enables company users to update their organizational details, including company name, industry, description, Contact information, and branding assets such as logos and media. It serves as the central module for rendering the editable profile UI, validating fields, and synchronizing profile data with the backend. The component also ensures that updates are reflected across the system, maintains consistency between profile information and job postings.

By breaking the profile logic into modular parts such as Profile Page, Profile Form, Profile Hook, and Profile Service, the architecture ensures that UI, business logic, validation, and data storage concerns remain separated. This decision will improve maintainability, reduce coupling, and make it easier to extend the feature with additional fields or new profile sections in future iterations.

Incoming Communication: The **Company Profile component** receives authenticated access from the **Security container**, which ensures that only users with the company role can update profile information. The **Browser Router** directs the user to the **Profile Page**, while **State Management** provides the current company data required to prefill editable fields such as name, industry, description, about us, and logo URLs. **Reusable Components** supply shared UI elements for rendering buttons, inputs, and upload controls within the **Profile Form**.

Outgoing Communications: When the user submits updates, the **Profile Form** sends all field values and media files to the **Profile Hook**, which communicates with backend REST APIs via the HTTP Unit. These require call endpoints for updating profile text fields, uploading logos, and posting media assets, while the **URL Config container** provides endpoint mapping and authorized routes. On the backend, the **Profile Service** delegates responsibilities to the **Validation Service** for field checking and to the **Media Storage Service** for processing and storing images or videos. Successful updates are written to the database through the **Company Repository** and returned to the frontend, where the **Profile Hook** updates **State Management** to refresh the UI with the latest company profile information.

5.4. Frontend Subscription C4 Component Diagram

Frontend C4 Modularized Subscription Feature

Overview: The diagram illustrates how the Job Manager frontend manages subscription plans, billing cycles, premium access, and payment-related actions for company users. The diagram shows how UI-level components, logic hooks, and backend-facing services collaborate to support subscription upgrades, renewals, cancellations, and payment processing. It also highlights interactions with the URL Config, HTTP utilities, State Management, and external third-party payment services like Stripe, ensuring the entire subscription workflow integrates smoothly with backend APIs and external billing providers.

Purpose: The subscription feature enables company users to manage their subscription lifecycle, including selecting plans, choosing billing cycles, reviewing pricing, and performing actions such as upgrading, downgrading, renewing, pausing, or cancelling subscriptions. The feature is responsible for presenting all subscription information in an editable UI, validating input fields, and synchronizing subscription state with backend

services. Its modular design, consisting of **Subscription Form**, **Subscription Quick Actions**, **useSubscriptionHook**, and **Subscription Service**, ensures a clear separation between UI rendering, business logic, API communication, and data handling. This separation improves maintainability and allows future enhancements, such as new payment methods or additional subscription tiers, to be introduced without disrupting existing modules.

Incoming Communications: The subscription feature receives authenticated access and user role verification from the **Security Container**, ensuring that only company users can manage subscription plans. The **Browser Router** navigates users to the subscription page, while the **Page container** renders the base layout for all subscription-related interfaces. The **State Management** container provides the current subscription status and user sessions. Information and premium flags are needed to initialize the UI. The **Assets container** supplies icons and static files displayed within the **Subscription Form**, and the **Reusable Components** container provides shared UI elements (buttons, dropdowns, inputs) used to build both the **Subscription Form** and **Subscription Quick Actions**.

Within the feature itself, the **Subscription Form** and **Subscription Quick Actions components** send all user inputs (plan changes, billing cycle selections, cancellation requests, renewal actions) into the **useSubscriptionHook**, which acts as the central logic layer. On the backend side, the **Subscription Service** may also receive contextual information from third-party payment systems, such as login or transaction states, before communicating with **Subscription APIs**. The **URL Config** container is used during incoming requests to determine permitted endpoint routes and credential expectations.

Outgoing Communications: When users submit changes (selecting new plans, renewing, cancelling, upgrading), the **Subscription Form** and **Quick Actions components** pass these inputs to the **useSubscriptionHook**, which validates the form data, constructs the appropriate request payload, and determines which subscription action to execute. The hook then makes outbound REST API calls through the **HTTP Util container**, which attaches authentication headers, parses metadata, and uses endpoint mapping defined in the **URL Config**. These outgoing requests trigger backend operations such as createSubscription, updateSubscription, renewSubscription, cancelSubscription, and the retrieval of invoices or billing details.

The **Subscription Service** receives API responses, interprets them, and maps them to frontend models before sending the data back to the hook. Once processed, the hook

updates the **State Management container**, ensuring the UI instantly reflects the latest subscription status, billing state, or plan tier. For payment-related actions, the frontend may also redirect users to external third-party payment systems, and once payment is completed or cancelled, the results are passed back to the **useSubscriptionHook** for further processing and state updates. These outgoing communications ensure that the subscription workflow stays synchronized across the frontend, backend, and external payment providers.

5.5. Frontend Applicant Search C4 Component Diagram

Frontend C4 Modularized Applicant Search Feature

Overview: The **Applicant Search Feature** is built with a modular approach to ensure the UI is clean and the logic is reusable. The **Applicant Search Page** component renders the overall search interface, while the **Applicant Search Form** component displays the input fields for search criteria like skills, job type, and experience. Once the user submits their search query, the input data is passed to the **Applicant Search Hook**, which is responsible for the core logic, including validating inputs, preparing the search query, and triggering the applicant search process.

Purpose: The Applicant Search Feature allows company users to search for applicants based on specific criteria such as skills, job type, experience, and location. The feature is designed to be flexible and modular, allowing users to easily find applicants that match their requirements. It provides efficient search experience while maintaining clear separation between the UI, business logic, and backend communication.

Incoming communication:

1. Authentication and User Info:

- The **Applicant Search Feature** ensures that the user is authenticated and authorized to perform search actions by checking through the **Security** container.
- **State Management** provides the current user data, ensuring that the search page is pre-filled with any relevant user or profile data when needed.

2. Search Criteria Validation:

- Before sending the search query to the backend, the **Search Criteria Validation Service** checks the input data to make sure it is valid and complete.

- Once the search criteria pass validation, they are forwarded to the **Applicant Search Service** for querying the backend.

Outgoing communication:**1. Search Query Submission:**

- The **Applicant Search Form** collects the search criteria and sends the data to the **Applicant Search Hook**, which prepares the query and calls the **Applicant Search Service**.
- The **Applicant Search Service** interacts with the backend API to retrieve applicants that match the search criteria.

2. Backend Processing:

- The **Applicant Search Service** handles the backend query and processes the results, mapping the applicant data into a format suitable for the frontend UI.
- Communication with the backend is managed through **HTTP Util** and **URL Config**, which ensure that the correct API routes, headers, and authentication are used.

3. UI Update and Display:

- The search results are returned to the UI, where they are displayed for the user.
- **State Management** stores any necessary user or profile data, while **Reusable Components** such as buttons and filters are updated to maintain consistency throughout the application.

Interaction Flow:

- UI collects search criteria → **Applicant Search Hook** validates and prepares data → **Search Criteria Validation Service** checks input → **Applicant Search Service** queries backend → results returned and displayed on the UI.

6. Architecture Rationale

6.1. Data Model Justification

Maintainability:

- **Advantages:** The data model is divided into clear functional domains: Applicant, Company, JobPost, Subscription, Authentication, which makes it easier for team members to understand and update each part independently. The normalized structure reduces redundancy, so maintaining or updating one table does not unintentionally affect others. This modularity supports easier debugging and consistent testing workflows across squads.
- **Disadvantages:** The high degree of normalization produces many interconnected entities, which increases the number of relationships/joins required to retrieve complete objects, new developers may find it more difficult to understand the entire schema due to its size and interdependencies. Complex join queries also require careful maintenance to avoid performance degradation.

Extensibility:

- **Advantage:** New features can be added easily because each domain/module is separated. For example, adding applicant certifications, new subscription tiers, or extra company fields can be done without changing the rest of the system. This fits well with DEVision's multi-team development, where different squads build different features.
- **Disadvantage:** Over time, adding too many small entities may clutter the schema. The squad must coordinate properly to avoid creating duplicated or overlapping entities, which requires good documentation and communication.

Resilience:

- **Advantage:** Separating high-activity entities (such as JobApplication and Notification) from core data helps the system continue working even if one part fails. For example, if notifications break, job posting and applicant profiles still work normally. This prevents total system failure.
- **Disadvantage:** Some entities, such as UserAuth or Subscription, are critical. If they fail, large parts of the system will stop functioning. The data model alone cannot prevent these issues and needs additional support such as caching or retries.

Scalability:

- **Advantage:** The model is designed so that high-volume entities can be scaled separately, for example through sharding or read replicas in future milestones.

Using simple numeric primary keys also makes indexing and searching fast. This helps DEVision handle more users and more job applications over time.

- **Disadvantage:** Complex joins/relationships across many entities can slow down the system when traffic grows. Without caching or denormalization, some queries may become too heavy under high load. Scaling relational databases also becomes harder at very large sizes.

Security:

- **Advantage:** Sensitive data is stored in dedicated entities such as UserAuth or CompanyAuth, which separates authentication information from profile data. This reduces the change of accidental exposure and supports strict role-based access control. Each module can have its own security rules.
- **Disadvantage:** The data model itself does not provide encryption or access control. These must be implemented at the API and backend layers. If APIs are not designed carefully, joins/relationships involving sensitive data may expose more information than intended.

Performance:

- **Advantage:** By splitting data into focused entities, most simple operations like updating a job post or editing a profile can run quickly. Indexed foreign keys make it fast to look up related data such as company job posts or applicant information. This keeps everyday interactions smooth for users.
- **Disadvantage:** When the system needs to gather detailed information (for example, full applicant profiles with skills, education, training, and experience), it must join multiple entities. These multi-entity queries can slow down performance as the database grows.

6.2. Frontend Architecture Justification

Maintainability:

- **Advantages:**
 - **Modular design** allows for easy updates and fixes. Components like Job Post Form and Applicant Search can be updated independently. For example, when fixing a bug inside the Job Post Form, the Applicant

Search module does not need to be touched, ensuring minimal impact on other features.

- **State Management** centralizes user data, making it easier to track and maintain. For example, when updating the user's session, State Management ensures that changes are reflected across different components like Job Post and Application Search without needing to update each individually.
- **Disadvantages: Complexity** increases as more components are added, which might make navigation and debugging more challenging for developers over time. For example, when adding a new Subscription Plan feature, developers must ensure it integrates smoothly with both the Job Post and Applicant Search features, which may introduce additional complexity.

Extensibility:

- **Advantages:**
 - **New features** (such as more filters or job post options) can be added without affecting existing components. The modular structure ensures flexibility.
 - **Reusable components** like the **Login Form** and **Search Bar** allow for consistent feature expansion.
- **Disadvantages: Feature overload** can lead to increased system complexity, making it harder to ensure all parts work seamlessly together as the app grows.

Resilience:

- **Advantages:**
 - **Error handling** ensures smooth user experience even when a failure occurs (e.g., failed login or job post).
 - **State Management** maintains a responsive UI by ensuring data updates are handled without crashing the app.
- **Disadvantages:** The system's **resilience depends on the backend**. If the backend service fails, the frontend can only handle the error to a certain extent, possibly affecting user experience.

Scalability:

- **Advantages:**
 - **Lazy loading** and **modular components** help the app perform well even with increasing users, as only necessary components are loaded.
 - **State Management** ensures smooth data handling as user demand grows.
- **Disadvantages: Memory consumption** could increase as more components and features are added, leading to performance issues if not optimized.

Security

- **Advantages:**
 - **Token-based authentication** and **role-based access control** ensure secure login and protect user data.
 - **HTTP Util** ensures secure communication between the frontend and backend with proper headers and token management.
- **Disadvantages:** Frontend vulnerabilities remain, and the system still relies on the backend for full security validation (e.g., protecting sensitive data).

Performance:

- **Advantages:**
 - Efficient state handling and lazy loading minimizes unnecessary data fetching, ensuring faster page loads.
 - Component-based architecture ensures that only the necessary parts of the UI are updated, reducing overhead and improving responsiveness.
- **Disadvantages:** Complex components may introduce performance overhead, especially with large data sets or high user traffic, requiring optimization.

6.3. Backend Architecture Justification

Maintainability

- **Advantages – Isolated modules:** Since modular monolith is the chosen architectural style, the separated modules encourage module boundaries and the usage of the same technology stack. Internal APIs allow the code to be understood, tested, and refactored more easily. For example, when fixing a bug inside the Subscription module, the Company module or the Auth module does not need to be touched.

- **Disadvantages – Limited add-ons:** Since the backend is a singular deployable unit, over time, as the codebase grows, the code can get harder to manage and understand, and testing will take more time. For example, when new features are added to modules like Subscription or Job Post, it will become increasingly more difficult for both current and future developers to understand.

Extensibility

- **Advantages - Loose coupling modules:** New features can be added by creating a new module or extending an existing one with minimal changes to other modules. For example, when adding a new subscription type for the Subscription module, the feature can just be introduced by extending the Subscription module without modification of other modules.
- **Disadvantages – Require total redeployment:** Adding a new feature may require the entire backend to be redeployed. This makes extensibility slower since a module cannot be deployed independently. For example, when adding the new feature as above in the Subscription module, redeploying only the Subscription module is not possible, but the entire backend must be redeployed.

Resilience

- **Advantages – Independent services:** When external systems such as Stripe or Google OAuth experience an outage, the backend will still function as normal. Companies will not be able to pay with Stripe or login with Google but will still be able to login using their own registered accounts and use other features of the system as normal.
- **Disadvantages – Memory leaks:** A catastrophic failure such as a bug or an out of memory state can bring down several modules. For example, a serious bug in the Company module causes the backend to run out of memory, which will result in a crash, and other modules are unavailable until the entire backend has restarted.

Scalability

- **Advantages – Distributed Traffic:** When there is an increase in user demand, the backend can run more instances. A load balancer will then be used to distribute traffic across those instances so that a single instance will not be overwhelmed. For example, when many companies are doing applicant searches at the same

time, a load balancer such as the AWS ALB can distribute search requests across all instances.

- **Disadvantages – No single scaling:** In the case that only a single module needs scaling, this would not be possible since the entire backend needs to be scaled altogether. For the same example as above, the backend cannot only scale the Job Post module; every other unrelated module also has to be scaled.

Security

- **Advantages – Global filters:** Global policies and input validation are done through Filter, then Security and Auth issues tokens and verify them. A sensitive operation such as paying for the premium subscription is only allowed for authorized and authenticated company users. For example, when a company is making a premium subscription payment, the request is sent to Filter and Security to verify tokens and access roles. If the token is invalid, the user will not be authenticated, or the user does not have the access role; in both cases the request will be blocked before reaching Payment or Subscription.
- **Disadvantages – Shared vulnerability:** Since the backend is a single deployable unit, a single security vulnerability can affect many modules. For example, if the Auth module somehow gets the secret key leaked, all modules that depend on Auth will become unsafe.

Performance

- **Advantages – Fast communication:** As the backend is a single application, modules can call each other directly internally, which is fast. Filter can also enforce pagination and keep the responses small and efficient by trimming out unnecessary data.
- **Disadvantages – Large resource demands:** Every module shares the same computing resources (CPU and memory), which means that if one module consumes a large amount of those resources, others will slow down. For example, many companies log in at once, so the Auth module will use much more resources than other modules, which in turn will cause those other modules to slow down.

7. Conclusion

The system provides a complete feature set for Job Managers and Job Applicants, such as companies to post and apply for jobs, company and applicant profile management, premium subscriptions, payment processing, and applicant search. The architecture consists of a modular React frontend and a modular Spring Boot monolith backend to ensure separation of concerns at the system level.

The architecture has several advantages. The modular monolithic backend design supports maintainability with well-defined modules, fast internal communication and consolidated security enforcement. It is also independent from external services such as Google OAuth, ensuring continuing operation in such a scenario. Meanwhile, a modularised frontend enables the reuse of components, scalable UI development and consistent cross-page state handling. Lazy loading ensures a responsive website for the end user.

However, the architecture has its limitations. The modular monolithic approach requires full redeployment when a single module is modified and gets harder to manage as the codebase grows. The system also cannot scale only to each feature's need, which is wasteful, and performance might be reduced drastically in case a module is overloaded. Additionally, the lack of database sharding may limit future scalability, and security vulnerabilities in shared backend components may affect multiple modules. The frontend can also grow highly complex and resource-intensive as features are added, and its dependency on backend availability can negatively affect user experience during backend failures.